

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e3b53aed-f4e\agent-a9334b383e540bd4d.jsonl`
- SHA-256: `ee40969f8e7e42d91aa2e7fb552bc641edb229c350d0725b6cd532c2648b6c87`
- Source modified: `2026-06-18T02:04:05+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_e3b53aed-f4e`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T02:00:03.300Z)

Review a bug-fix PR in badminton-highlight-indexer. Work READ-ONLY in this git worktree (do NOT modify): C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap

First run `git -C "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap" diff origin/master..HEAD` to see the exact change, then read the touched files + related code/tests. PR #213 fixes two standalone scripts (scripts/run_phase3_eval.py, scripts/run_process_and_stitch.py) that called `len()/truthiness` on a raw segmenter Result (Success|Failure dataclass, no `__len__`) ? crashing under the default 'motion' segmenter. Adds backend/results.unwrap_or_failure(result) -> (value, failure) (DRY of the backend/main.py inline unwrap) + tests/test_results.py.

Be adversarial ? try to find a real reason NOT to merge. Concrete, verifiable findings only (cite file:line); no style noise. severity must-fix only for a genuine bug/regression/break. blocks_merge=true ONLY on a real must-fix. Return the structured object.

LENS ? edge cases: partial states, None/empty, mixed inputs, exception types (is OperationalError message text stable across sqlite versions?), the case where a DIFFERENT OperationalError is raised, an empty Success(value=[]) flowing to the empty-guard.

2. assistant (2026-06-18T02:00:05.237Z)

I'll review this bug-fix PR. Let me start by examining the exact change.

3. user (2026-06-18T02:00:06.767Z)

```
diff --git a/backend/results.py b/backend/results.py
index d6b1ea4..75120c5 100644
--- a/backend/results.py
+++ b/backend/results.py
@@ -18,7 +18,7 @@ This module has no dependencies on the rest of the app.
 from __future__ import annotations

 from dataclasses import dataclass, field
-from typing import Generic, TypeVar, List, Optional, Any, Union
+from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union

 T = TypeVar("T")

@@ -58,3 +58,21 @@ class Failure:
 # Result[ValueType] means Success[ValueType] | Failure
 # (the Failure "error" type is fixed for this contract)
 Result = Union[Success[T], Failure]
```

```

+
+
+def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
+    """Split a segmenter/provider return into ``(value, failure)`` so a caller branches once.
+
+    - ``Failure`` -> ``(None, failure)``
+    - ``Success(value=v)`` -> ``(v, None)``
+    - a bare value (legacy code that predates this contract and still returns a raw list)
+    -> ``(value, None)``
+
+    This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline; callers
+    that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
+    len()``
+    on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
+    heterogeneous
+    (Success | Failure | legacy bare value)."""
+    if isinstance(result, Failure):
+        return None, result
+    value = result.value if hasattr(result, "value") else result
+    return value, None
diff --git a/scripts/run_phase3_eval.py b/scripts/run_phase3_eval.py
index 56e21ca..dd1263e 100644
--- a/scripts/run_phase3_eval.py
+++ b/scripts/run_phase3_eval.py
@@ -34,6 +34,7 @@ from backend.config import load_config
 from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
 # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
 from backend.pipeline.segmenters import segmenter_registry
+from backend.results import unwrap_or_failure
 from backend.eval.annotations import load_annotations
 from backend.eval.harness import evaluate, format_report_text, report_to_dict
 from backend.eval import batch
@@ -142,7 +143,15 @@ def main(argv=None) -> int:
     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
     db.add_video(video_id, video_path, "processed", [])
     t0 = time.time()
-    segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
+    # process_video returns a Result (Success|Failure) for some segmenters
+    # (e.g. the default `motion`); unwrap before len() so this doesn't crash with
+    # TypeError on a Success dataclass ? mirrors the unwrap in backend/main.py.
+    segs, failure = unwrap_or_failure(
+        segmenter.process_video(video_id, video_path, max_frames=args.max_frames))
+    if failure is not None:
+        print(f"        -> FAILED: {failure.message}")
+        skipped.append((vid, f"segment failed: {failure.message}"))
+        continue
     print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")

     gt = load_annotations(csv_path)
diff --git a/scripts/run_process_and_stitch.py b/scripts/run_process_and_stitch.py
index b8ffef6..ea7c2de 100644
--- a/scripts/run_process_and_stitch.py
+++ b/scripts/run_process_and_stitch.py
@@ -16,6 +16,7 @@ from backend.config import load_config
 from backend.utils.hashing import compute_video_id
 from backend.pipeline.segmenters import segmenter_registry
 from backend.pipeline.stitcher.compiler import VideoCompiler
+from backend.results import unwrap_or_failure
 from backend.tools.export import format_rally_summary

 logger = logging.getLogger(__name__)
@@ -78,7 +79,13 @@ def main():
     return
     print(f"Using segmenter: {seg_name}", flush=True)

```

```

        segmenter = segmenter_cls(db, cfg)
-       segments = segmenter.process_video(video_id, video_path)
+       # process_video returns a Result (Success|Failure) for some segmenters (e.g. `motion`);
+       # unwrap before len()/truthiness so this doesn't crash with TypeError on a Success
+       # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
+       segments, failure = unwrap_or_failure(segmenter.process_video(video_id, video_path))
+       if failure is not None:
+           print(f"[FAIL] Segmentation failed: {failure.message}")
+           return
+       print(f"Successfully indexed {len(segments)} play segments.", flush=True)

    if not segments:
diff --git a/tests/test_results.py b/tests/test_results.py
new file mode 100644
index 0000000..5b50767
--- /dev/null
+++ b/tests/test_results.py
@@ -0,0 +1,39 @@
+"""Unit tests for ``backend.results`` ? the Success/Failure contract + the
+``unwrap_or_failure``
+helper that standalone scripts use to avoid ``len()``-ing a raw ``Success`` dataclass."""
+import pytest
+
+from backend.results import Failure, Success, unwrap_or_failure
+
+def test_unwrap_success_returns_value_and_no_failure():
+    value, failure = unwrap_or_failure(Success(value=[{"start": 1.0}]))
+    assert value == [{"start": 1.0}]
+    assert failure is None
+
+def test_unwrap_failure_returns_failure_and_no_value():
+    f = Failure(message="Could not open video: x.mp4", is_recoverable=False)
+    value, failure = unwrap_or_failure(f)
+    assert value is None
+    assert failure is f
+
+def test_unwrap_bare_value_passes_through():
+    """Legacy segmenters that predate the Result contract still return a bare list ? the helper
+    passes those through unchanged as ``(value, None)`` so a mixed segmenter set all works."""
+    bare = [{"start": 1.0}, {"start": 2.0}]
+    value, failure = unwrap_or_failure(bare)
+    assert value == bare
+    assert failure is None
+
+def test_unwrap_fixes_the_len_on_success_crash():
+    """Regression for the scripts bug: ``len()`` on a raw ``Success`` raises ``TypeError``;
+    unwrapping first yields the underlying list so callers (run_phase3_eval /
+    run_process_and_stitch, which default to the ``motion`` segmenter) can ``len()`` it."""
+    result = Success(value=[1, 2, 3])
+    with pytest.raises(TypeError):
+        len(result) # the original bug (Success has no __len__)
+    segs, failure = unwrap_or_failure(result)
+    assert failure is None
+    assert len(segs) == 3 # fixed

```

4. assistant (2026-06-18T02:00:11.937Z)

Let me read the full files to understand context: the backend/results.py, the inline unwrap in backend/main.py it claims to mirror, and the touched scripts.

5. user (2026-06-18T02:00:12.430Z)

```
1      """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3      This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5      The goal is to make it impossible to confuse "no data / no rallies" with "something
6      actually failed"
7      (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
8
9      We use simple dataclasses so callers can use isinstance checks or structural pattern
10     matching.
11
12     This is intentionally small and incremental ? we start by applying it to the segmenter
13     and provider layers (the biggest sources of silent [] returns today).
14
15     The types are generic where it makes sense. Callers can still access .value on Success
16     or .message on Failure.
17
18     This module has no dependencies on the rest of the app.
19     """
20
21     from __future__ import annotations
22
23     from dataclasses import dataclass, field
24     from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union
25
26     T = TypeVar("T")
27
28     @dataclass
29     class Success(Generic[T]):
30         """Represents a successful outcome with an optional list of non-fatal warnings."""
31
32         value: T
33         warnings: List[str] = field(default_factory=list)
34
35         def is_success(self) -> bool:
36             return True
37
38         def unwrap(self) -> T:
39             """Return the value. Only safe to call if you already know it's a Success."""
40             return self.value
41
42     @dataclass
43     class Failure:
44         """Represents a failure with a human-readable message and optional details."""
45
46         message: str
47         details: Optional[dict[str, Any]] = None
48         is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
49
50         def is_success(self) -> bool:
51             return False
52
53         def unwrap(self) -> Any:
54             """Raise to make misuse obvious in code that assumes success."""
55             raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
56
57     # Convenient alias for use in type hints: Result[list[dict], Failure]
58     # Result[ValueType] means Success[ValueType] | Failure
59     # (the Failure "error" type is fixed for this contract)
60     Result = Union[Success[T], Failure]
61
```

```

62
63     def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64         """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66         - ``Failure``          -> ``(None, failure)``
67         - ``Success(value=v)`` -> ``(v, None)``
68         - a bare value (legacy code that predates this contract and still returns a raw
list)
69         -> ``(value, None)``
70
71         This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72         that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73         on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74         (Success | Failure | legacy bare value)."""
75         if isinstance(result, Failure):
76             return None, result
77         value = result.value if hasattr(result, "value") else result
78         return value, None
79

```

6. assistant (2026-06-18T02:00:15.102Z)

Now let me look at the inline unwrap in backend/main.py that this claims to mirror, and the full context of both scripts.

7. user (2026-06-18T02:00:16.550Z)

```

35:from backend.results import Failure # standardized error/result contract
192:     result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
195:     # state (same destroy-after-confirm contract as the Failure branch), then let
201:     if isinstance(result, Failure):
213:     segments = result.value if hasattr(result, "value") else result
220:     "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
234:     if paths and isinstance(response, dict):
347:def process_video(req: ProcessRequest):
600:     sc = meta.get("shot_count") if isinstance(meta, dict) else None
835:     result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
838:     if isinstance(result, Failure):
843:     segments = result.value if hasattr(result, "value") else result

```

8. user (2026-06-18T02:00:19.820Z)

```

188         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
189         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
190         segmenter = segmenter_cls(db_instance, global_config)
191         try:
192             result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
193         except Exception:
194             # A raising segmenter must not leave half-written rows: restore the pre-run
195             # state (same destroy-after-confirm contract as the Failure branch), then
let
196             # the job worker record the failure.
197             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
198             raise
199         segmentation_ran = True

```

```

200
201         if isinstance(result, Failure):
202             logger.error(f"[API] Segmenter failed: {result.message}")
203             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
204             response = {
205                 "video_id": video_id,
206                 "status": "failed",
207                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
208                 "results": [r.model_dump() for r in val_results],
209                 "play_segments": [],
210             }
211             return response
212
213         segments = result.value if hasattr(result, "value") else result
214         notify("finalizing")
215         _finalize_reingest(video_id, segments, play_wm, cand_wm)
216
217         response = {
218             "video_id": video_id,
219             "status": "success",
220             "message": f"Successfully indexed {len(segments)} play segments using
221 '{seg_name}' segmenter.",
222             "results": [r.model_dump() for r in val_results],
223             "play_segments": segments,
224         }
225         return response
226     finally:
227         # Always emit the per-video observability report (next to the video, or to
228         # report.output_dir). Never raises. Surface the paths in the response.

```

9. assistant (2026-06-18T02:00:23.128Z)

The helper faithfully mirrors the inline unwrap at main.py:201/213. Now let me check the full scripts to understand the empty-guard logic and what happens with the OperationalError path the lens mentions. Let me look at both scripts fully.

10. user (2026-06-18T02:00:23.652Z)

```

1     import os
2     import json
3     import logging
4     from dotenv import load_dotenv
5
6     load_dotenv()
7
8     # Central root-logger setup ? all backend modules use logging.getLogger(__name__).
9     # Quiet third-party HTTP chatter by default; re-honoured against the config knob
10    # (logging.verbose_http) once the config is loaded in main().
11    from backend.utils.logging_setup import setup_logging
12    setup_logging()
13
14    from backend.infrastructure.database import Database
15    from backend.config import load_config
16    from backend.utils.hashing import compute_video_id
17    from backend.pipeline.segmenters import segmenter_registry
18    from backend.pipeline.stitcher.compiler import VideoCompiler
19    from backend.results import unwrap_or_failure
20    from backend.tools.export import format_rally_summary
21
22    logger = logging.getLogger(__name__)
23
24    def main():
25        video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
26        output_dir = r"C:\Users\avidu\Videos\Testing"
27        output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"

```

```

28
29     print("Loading config...")
30     cfg = load_config() # typed AppConfig ? single source of defaults
31     # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter
quiet).
32     setup_logging(verbose_http=cfg.logging.verbose_http)
33
34     print("Verifying files exist...")
35     if not os.path.exists(video_path):
36         print(f"Error: Video file not found at {video_path}")
37         return
38
39     os.makedirs(cfg.output.output_dir, exist_ok=True)
40     db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
41     db = Database(db_path)
42
43     # Calculate MD5 of the video to uniquely identify it
44     import time
45     print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
46     t0 = time.time()
47     video_id = compute_video_id(video_path)
48     print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
49
50     segments = []
51     cached_video = db.get_video(video_id)
52     cache_hit = False
53
54     if cached_video and cached_video.get("status") == "processed":
55         segments = db.query_play_segments(video_id, {})
56         if len(segments) > 0:
57             print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
58             choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
59             if choice == '1':
60                 print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
61                 cache_hit = True
62             else:
63                 print("[CACHE] Ignoring cache...", flush=True)
64             else:
65                 print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
66
67     if not cache_hit:
68         print("[CACHE] Initializing clean indexing...", flush=True)
69         db.clear_video_data(video_id)
70
71     print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
72     db.add_video(video_id, video_path, "processed", [])
73
74     print("Running segmenter & indexing...", flush=True)
75     seg_name = cfg.indexing.default_segmenter
76     segmenter_cls = segmenter_registry.get(seg_name)
77     if not segmenter_cls:
78         print(f"[FAIL] Segmenter '{seg_name}' not found.")
79         return
80     print(f"Using segmenter: {seg_name}", flush=True)
81     segmenter = segmenter_cls(db, cfg)
82     # process_video returns a Result (Success|Failure) for some segmenters (e.g.
`motion`);
83     # unwrap before len()/truthiness so this doesn't crash with TypeError on a
Success

```

```

84         # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
85         segments, failure = unwrap_or_failure(segmenter.process_video(video_id,
video_path))
86         if failure is not None:
87             print(f"[FAIL] Segmentation failed: {failure.message}")
88             return
89         print(f"Successfully indexed {len(segments)} play segments.", flush=True)
90
91     if not segments:
92         print("No play segments detected. Cannot compile highlights.")
93         return
94
95
96     end_padding = cfg.stitching.end_padding
97     begin_padding = cfg.stitching.begin_padding
98     min_shot_count = cfg.stitching.min_shot_count
99
100    if cache_hit:
101        print("\n--- Stitching Customizations ---")
102        try:
103            bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
104            if bpad_input:
105                begin_padding = float(bpad_input)
106
107            pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
108            if pad_input:
109                end_padding = float(pad_input)
110
111            shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
112            if shot_input:
113                min_shot_count = int(shot_input)
114        except ValueError:
115            print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")
116
117    print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)
118
119    # Filter segments where shot_count >= min_shot_count
120    time_pairs = []
121    skipped_count = 0
122    for s in segments:
123        meta = s.get("metadata", {})
124        if isinstance(meta, str):
125            try:
126                meta = json.loads(meta)
127            except Exception:
128                meta = {}
129        shot_count = meta.get("shot_count", 0)
130        if shot_count >= min_shot_count:
131            time_pairs.append((s["start_time"], s["end_time"]))
132        else:
133            skipped_count += 1
134
135    if skipped_count > 0:
136        print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.", flush=True)
137    print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
flush=True)
138
139    # End-of-run rally table for manual QA ? sourced from the produced play_segments so

```

```

140     # it lines up row-for-row with the rally CSV. Logged as one block (single timestamp,
141     # aligned rows) so it survives even if the table is the last thing the owner scans.
142     def _log_rally_summary() -> None:
143         logger.info("\n%s", format_rally_summary(segments))
144
145     if not time_pairs:
146         print("No play segments passed the filtering criteria. Cannot compile
highlights.")
147         _log_rally_summary()
148         return
149
150     compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding,
151                               watermark=cfg.stitching.watermark)
152     try:
153         out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
154         print(f"Stitched highlight video saved to: {out_path}", flush=True)
155     except Exception as e:
156         print(f"Error during stitching: {e}", flush=True)
157
158     _log_rally_summary()
159
160 if __name__ == "__main__":
161     main()
162

```

11. assistant (2026-06-18T02:00:29.323Z)

Now the empty-guard case the lens mentions: `Success(value=[])` flowing to `if not segments` at line 91. The helper returns `([], None)`, so `segments = []`, `failure is None`, prints "Successfully indexed 0 play segments", then line 91 `if not segments` catches it and returns "No play segments detected." That's correct behavior. Let me now look at run_phase3_eval.py around the touched area.

12. user (2026-06-18T02:00:30.547Z)

```

110     for e in entries:
111         if args.only and e["video_id"] not in args.only:
112             continue
113         targets.append(e)
114     if args.limit:
115         targets = targets[: args.limit]
116
117     per_video = []
118     skipped = []
119     print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")
120
121     for e in targets:
122         vid = e["video_id"]
123         video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
124         csv_path = os.path.join(manifest_dir, e["csv"])
125
126         if not video_path or not os.path.exists(video_path):
127             skipped.append((vid, "no video file"))
128             print(f"[skip] {vid}: no video file ({e.get('local_path')})")
129             continue
130         if not os.path.exists(csv_path):
131             skipped.append((vid, "no GT csv"))
132             print(f"[skip] {vid}: GT csv missing ({csv_path})")
133             continue
134
135         video_id = get_file_md5(video_path)
136
137         # Segment unless score-only or already present (and not --reprocess).

```

```

138         if not args.score_only:
139             already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
140             if already and not args.reprocess:
141                 print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
142             else:
143                 print(f"[proc] {vid}: segmenting with {args.segmenter}...")
144                 db.add_video(video_id, video_path, "processed", [])
145                 t0 = time.time()
146                 # process_video returns a Result (Success|Failure) for some segmenters
147                 # (e.g. the default `motion`); unwrap before len() so this doesn't crash
with
148                 # TypeError on a Success dataclass ? mirrors the unwrap in
backend/main.py.
149                 segs, failure = unwrap_or_failure(
150                     segmenter.process_video(video_id, video_path,
max_frames=args.max_frames))
151                 if failure is not None:
152                     print(f"          -> FAILED: {failure.message}")
153                     skipped.append((vid, f"segment failed: {failure.message}"))
154                     continue
155                     print(f"          -> {len(segs)} segments in {time.time() - t0:.0f}s")
156
157                 gt = load_annotations(csv_path)
158                 report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
159                 print(format_report_text(report, show_failures=args.show_failures))
160                 print()
161                 d = report_to_dict(report)
162                 d["collector_video_id"] = vid
163                 per_video.append(d)
164
165                 agg = batch.aggregate(per_video, stages)
166                 print(batch.format_aggregate(agg))
167                 if skipped:
168                     print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
169
170                 if args.json_out:
171                     with open(args.json_out, "w") as f:
172                         json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
173                             "aggregate": agg, "per_video": per_video,
174                             "skipped": skipped}, f, indent=2)
175                     print(f"\nJSON report written to {args.json_out}")
176
177                 return 0
178
179
180     if __name__ == "__main__":
181         raise SystemExit(main())
182

```

13. assistant (2026-06-18T02:00:35.214Z)

In `run_phase3_eval.py` the unwrapped ``segs`` is only used for the print at line 155 ? the actual evaluation reads from the DB (line 158 ``evaluate(db, video_id, ...)``), so the segments value isn't load-bearing there beyond logging. Fine.

Now let me investigate the lens-specific concern about `OperationalError`. The lens mentions "is `OperationalError` message text stable across sqlite versions?" and "the case where a `DIFFERENT OperationalError` is raised". This suggests there's exception-string matching somewhere. Let me search the touched scripts and the unwrap path for any `OperationalError` handling. This may relate to how ``process_video`` raises vs returns.

14. user (2026-06-18T02:00:37.211Z)

```
backend\infrastructure\database.py:40:         except sqlite3.OperationalError:
```

15. user (2026-06-18T02:00:39.835Z)

```
25         yield conn
26     finally:
27         conn.close()
28
29     def _get_connection(self):
30         # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31         # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32         conn = sqlite3.connect(self.db_path, timeout=30.0)
33         conn.row_factory = sqlite3.Row
34         # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
35         conn.execute("PRAGMA foreign_keys = ON")
36         # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
37         conn.execute("PRAGMA busy_timeout = 30000")
38         try:
39             conn.execute("PRAGMA journal_mode = WAL")
40         except sqlite3.OperationalError:
41             pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
42         return conn
43
44     def _init_db(self):
45         """Initializes tables for videos, play_segments, and events."""
46         with self._connect() as conn:
47             cursor = conn.cursor()
48
49             # Videos Table
50             cursor.execute("""
51                 CREATE TABLE IF NOT EXISTS videos (
52                     id TEXT PRIMARY KEY,
53                     filepath TEXT NOT NULL,
54                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

16. assistant (2026-06-18T02:00:48.358Z)

This `OperationalError` handling is pre-existing, not touched by this PR, and it's a broad ``except ... : pass`` (no message-text matching), so the lens's concern about message-text stability doesn't apply to anything this PR changes. This is unrelated to the `unwrap` helper. The lens was a hypothesis to check; verified it's not relevant to this PR's diff.

Now let me focus on the real adversarial concern with the helper logic itself. The key line is:

```
`python
value = result.value if hasattr(result, "value") else result
`
```

This uses ``hasattr(result, "value")`` for a bare ``Success``. Let me consider edge cases:

1. `***`Success(value=None)`**` ? ``hasattr`` is `True`, returns ``(None, None)``. Caller checks ``if failure is not None`` ? fine, then ``len(None)`` would crash. But a segmenter returning ``Success(value=None)`` is a contract violation; `main.py`'s inline version has identical behavior. Not a regression.

2. `***A bare value that happens to have a `.value` attribute**` ? e.g. a numpy scalar, or some object with ``.value``. But segmenters return ``list[dict]``; lists don't have ``.value``. Acceptable given documented contract.

3. `***A `Failure`-like duck type**` ? ``isinstance(result, Failure)`` is strict. Fine.

Let me check the ``run_all_tests.py`` includes the new test, and verify the test file and whether

tests actually pass. Let me also confirm there are no other callers in scripts/ that still have the raw bug (i.e., is the fix complete across the codebase?).

17. user (2026-06-18T02:00:49.400Z)

```
scripts\run_process_and_stitch.py:82:      # process_video returns a Result (Success|Failure)
for some segmenters (e.g. `motion`);
scripts\run_process_and_stitch.py:85:      segments, failure =
unwrap_or_failure(segmenter.process_video(video_id, video_path))
scripts\run_phase3_eval.py:146:      # process_video returns a Result
(Success|Failure) for some segmenters
scripts\run_phase3_eval.py:150:      segmenter.process_video(video_id, video_path,
max_frames=args.max_frames))
```

18. assistant (2026-06-18T02:00:52.064Z)

Both script call sites are fixed. Let me check there are no OTHER scripts/tools calling `process\_video` raw that would also have the bug but were missed (scope completeness), and confirm the motion segmenter actually returns a Success (to validate the bug premise).

19. user (2026-06-18T02:00:53.199Z)

```
Found 30 files
backend\results.py
scripts\run_process_and_stitch.py
scripts\run_phase3_eval.py
tools\generate_highlights.py
tests\test_yolo_hybrid.py
tests\test_windowing.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_tracknet_hybrid.py
tests\test_job_worker_loop.py
tests\test_generate_highlights.py
tests\test_gemini.py
tests\test_fusion_hybrid.py
tests\test_base.py
tests\test_api_endpoints.py
tests\test_async_jobs.py
mypy.ini
docs\archives\checkpoints\2026-06-logging-reporting\README.md
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\QUALITY_ITERATIONS.md
docs\PER_VIDEO_OUTPUT_LAYOUT.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\CODE_MAP.md
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\base.py
backend\main.py
backend\eval>windowing.py
backend\eval\served_gate_a.py
```

20. assistant (2026-06-18T02:00:56.147Z)

There's `tools/generate\_highlights.py` which also calls `process\_video`. Let me check whether it has the same raw `len()/truthiness` bug? if so, this PR's fix is incomplete (same class of bug left unfixed). Let me also confirm the motion segmenter returns Success to validate the bug premise.

21. user (2026-06-18T02:00:57.488Z)

```
42:from backend.results import Failure
```

```

116:         len(video_paths), out_dir, segmenter, force)
129:         logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
134:         i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
144:         logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
167:         result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
168:         if isinstance(result, Failure):
175:             logger.info("rallies detected: %d", len(pairs))
200:             logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
225:         logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
249:         if len(args.detect_from) != len(args.video):
250:             raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
251:                             f"--video count ({len(args.video)}); pair them 1:1 in order")

```

22. assistant (2026-06-18T02:01:00.479Z)

`tools/generate\_highlights.py` already handles the Result correctly (line 167-168: `isinstance(result, Failure)` check) ? it's a third inline copy of the same unwrap. Let me confirm it unwraps `.value` too.

23. user (2026-06-18T02:01:01.190Z)

```

165         # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
166         # main.py); cast keeps the type checker honest about that contract.
167         result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid,
proc_detect)
168         if isinstance(result, Failure):
169             logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
170             summary["failed"].append(name)
171             continue
172
173         segs = db.query_play_segments(vid, {})
174         pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
175         logger.info("rallies detected: %d", len(pairs))
176         if not pairs:
177             logger.warning("no rallies -> no reel")
178             summary["no_rallies"].append(name)
179             continue

```

24. assistant (2026-06-18T02:01:07.486Z)

`generate\_highlights.py` is a fourth inline copy (it reads segments back from the DB rather than using the return value, so it doesn't need `.value`). It's not buggy, but it's a missed DRY opportunity ? a nit, not a must-fix, and arguably out of scope since it doesn't crash.

Now let me validate the bug premise and run the actual tests. Let me confirm the motion segmenter returns a `Success` (so the scripts genuinely crashed before this fix) and run the new test plus a broader test run.

25. user (2026-06-18T02:01:09.510Z)

```

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-
unwrap/backend/pipeline/segmenters/motion.py:20:     def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-
unwrap/backend/pipeline/segmenters/motion.py:31:         return Failure(message=f"Could not
open video: {video_path}", is_recoverable=False)
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-
unwrap/backend/pipeline/segmenters/motion.py:224:         return Success(value=segments_data)
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-
unwrap/backend/pipeline/segmenters/base.py:24:     def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) ->

```

```
"Result[List[Dict[str, Any]]":
```

26. assistant (2026-06-18T02:01:13.616Z)

Bug premise confirmed: `motion.py` returns `Success(value=segments_data)` (line 224) or `Failure` (line 31). The old `len(segs) / if not segments` on a raw `Success` would crash with `TypeError`. Note also `motion.py:20` has a stale return annotation (`-> List[Dict[str, Any]]`) that contradicts its actual `Success` / `Failure` returns ? pre-existing, out of scope.

Now let me run the new test and a broader test sweep to verify green.

27. user (2026-06-18T02:01:15.598Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 4 items

tests/test_results.py::test_unwrap_success_returns_value_and_no_failure PASSED [ 25%]
tests/test_results.py::test_unwrap_failure_returns_failure_and_no_value PASSED [ 50%]
tests/test_results.py::test_unwrap_bare_value_passes_through PASSED [ 75%]
tests/test_results.py::test_unwrap_fixes_the_len_on_success_crash PASSED [100%]

===== 4 passed in 0.10s =====
```

28. assistant (2026-06-18T02:01:18.578Z)

All 4 new tests pass. Let me verify the two scripts at least import cleanly (no import errors / syntax issues from the new import), and run a quick sanity import. Then run the broader suite to confirm no regression.

29. user (2026-06-18T02:01:21.291Z)

```

parse OK
([], None)
(None, Failure(message='x', details=None, is_recoverable=False))
([1, 2], None)

```

30. assistant (2026-06-18T02:01:24.645Z)

Empty `Success(value=[])` correctly yields `([], None)` ? flows to the empty-guard as a list, exactly as intended. Now let me run the broader test suite to confirm no regression (focus on the scripts/results area plus a representative sweep).

31. user (2026-06-18T02:02:43.367Z)

```
def test_gate_a_never_emptyies_a_video():
    """NEVER-EMPTY safeguard: if the crossings threshold would drop EVERY window, the gate
       falls back to the full ungated set rather than hand off zero rallies. Here min_crossings
       is set absurdly high so neither window clears it ? yet both still reach the handoff."""
> db, provider, _res = _run(indexing={"fusion": {"min_crossings": 999}})
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

tests\test_fusion_hybrid.py:102:
-----
tests\test_fusion_hybrid.py:55: in _run
    res = seg.process_video("v1", "clip.mp4")
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
backend\pipeline\segmenters\trajectory_hybrid.py:273: in process_video
    os.makedirs(handoff_dir, exist_ok=True)
```

name = 'output\\chunks_gemini_handoff', mode = 511, exist_ok = True

> ???

E FileExistsError: [WinError 183] Cannot create a file when that file already exists:
'output\\chunks_gemini_handoff'

<frozen os>:228: FileExistsError

===== short test summary info =====
FAILED tests/test_fusion_hybrid.py::test_gate_a_never_emptyies_a_video - FileE...
1 failed, 849 passed, 7 skipped in 74.34s (0:01:14)

32. assistant (2026-06-18T02:02:48.156Z)

One failure, but it's a pre-existing Windows-specific test isolation issue in
`test\_fusion\_hybrid.py` (a `FileExistsError` on `os.makedirs(handoff\_dir, exist\_ok=True)` ? a
known Windows quirk where `exist\_ok=True` still raises on a junction/dir-already-exists race),
entirely unrelated to this PR. Let me confirm it's pre-existing by checking it fails on
origin/master too ? quick targeted check by verifying the failing code path is untouched by the
diff. The diff only touched `backend/results.py`, two scripts, and the new test file ?
`trajectory\_hybrid.py` and `test\_fusion\_hybrid.py` are untouched. Let me confirm this failure
reproduces on master to be certain it's not introduced.

33. user (2026-06-18T02:02:56.403Z)

. [100%]
1 passed in 5.08s

34. assistant (2026-06-18T02:03:01.007Z)

Confirmed: the failure is a test-isolation/ordering artifact (passes in isolation, fails only
when run with the full suite due to a leftover `output/chunks\_gemini\_handoff` dir). It is
entirely unrelated to this PR ? the diff doesn't touch `trajectory\_hybrid.py` or
`test\_fusion\_hybrid.py`, and the failing line is `os.makedirs(..., exist\_ok=True)` cross-test
pollution. Not a blocker for this PR.

Now let me verify lint/type checks on the touched files match CI. Let me run ruff and mypy on
the changed files.

35. user (2026-06-18T02:03:03.356Z)

All checks passed!
--- mypy ---
Success: no issues found in 1 source file

36. assistant (2026-06-18T02:03:08.951Z)

Ruff clean, mypy clean on the new module. Let me check the mypy.ini to see whether scripts/ are
type-checked in CI (the scripts were in the mypy match list earlier), and run mypy on the
scripts too to be thorough.

37. user (2026-06-18T02:03:13.781Z)

backend\sports__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
scripts\run_phase3_eval.py:106: error: Argument 2 to "BasePlaySegmenter" has incompatible type
"AppConfig"; expected "dict[str, Any]" [arg-type]
scripts\run_phase3_eval.py:150: error: Item "None" of "BasePlaySegmenter | None" has no
attribute "process_video" [union-attr]
scripts\run_phase3_eval.py:155: error: Argument 1 to "len" has incompatible type "Any | None";
expected "Sized" [arg-type]

Found 3 errors in 1 file (checked 2 source files)

38. assistant (2026-06-18T02:03:16.559Z)

Mypy reports errors in `run\_phase3\_eval.py`. I need to determine whether these are pre-existing (and whether scripts are even in CI's mypy scope) or newly introduced by this PR. Let me check the mypy.ini config and whether line 155's `len(segs)` error is new (it touches the changed code).

39. user (2026-06-18T02:03:17.220Z)

```
1      [mypy]
2      # Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
3      # modules and pins an explicit quarantine list below ? not a typing crusade.
4      # Ratchet: when you touch a quarantined module, try removing its entry.
5      ignore_missing_imports = True
6      # backend/ is a namespace package (no root __init__.py) ? required so mypy
7      # maps files to backend.* module names instead of duplicating top-level names.
8      explicit_package_bases = True
9
10     # --- Quarantine (known-loose modules, each with a reason) ---
11
12     # Optional module globals (db_instance/global_config set only in main());
13     # properly fixed by the deferred app-factory/lifespan refactor (BACKLOG).
14     [mypy-backend.main]
15     ignore_errors = True
16
17     # Legacy demo segmenter (fabricated metadata; coverage-omitted too).
18     [mypy-backend.pipeline.segmenters.motion]
19     ignore_errors = True
20
21     # yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
22     # to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
23     # here is pre-existing orchestration debt unrelated to that move (implicit-Optional
24     # process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
25     # annotation) ? ratchet those out in a focused typing pass, not here.
26     # (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)
27     [mypy-backend.pipeline.segmenters.yolo_hybrid]
28     ignore_errors = True
29
30     [mypy-backend.pipeline.segmenters.gemini]
31     ignore_errors = True
32
33     # Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.
34     [mypy-backend.pipeline.segmenters.base]
35     ignore_errors = True
36
```

40. assistant (2026-06-18T02:03:21.622Z)

The mypy.ini quarantine list does NOT include `scripts.\*`. So the question is whether CI runs mypy over `scripts/`. Let me check how CI invokes mypy (the workflow file or run_all_tests), and whether these 3 errors are pre-existing on master vs introduced. Line 155 `len(segs)` is on the changed code, so it could be a newly-introduced error.

41. user (2026-06-18T02:03:23.213Z)

```
.github/workflows/ci.yml
=== mypy invocations in CI ===
.github/workflows/ci.yml:33: # Type gate (mypy.ini at repo root): lenient green baseline with
an explicit
.github/workflows/ci.yml:35: # up. Typed core deps are installed so mypy sees their real
signatures
.github/workflows/ci.yml:45:      - name: Install mypy + typed core deps
```

```
.github/workflows/ci.yml:46:      run: python -m pip install mypy pydantic fastapi uvicorn
python-dotenv google-genai numpy
.github/workflows/ci.yml:48:      run: mypy backend training
.github/workflows/ci.yml:87:      # (pytest, pytest-cov, httpx for the fastapi TestClient,
ruff, mypy).
```

42. assistant (2026-06-18T02:03:29.835Z)

Key finding: CI runs `mypy backend training` (ci.yml:48) ? it does **not** type-check `scripts/`. So the 3 mypy errors in `run\_phase3\_eval.py` are not gated by CI and are not a merge blocker. Two of them (lines 106, 150 ? the `AppConfig`/`None` issues) are pre-existing patterns unrelated to this PR. Line 155 `len(segs)` involves the changed code, but since scripts aren't in the mypy gate, it doesn't break CI.

Let me confirm line 155's error is genuinely caused by the new `Any | None` return type of `unwrap\_or\_failure` (worth noting as an observation), and verify `run\_process\_and\_stitch.py` mypy status (it wasn't in the checked output ? it imports cleanly). Let me also double check CI runs mypy on `backend/results.py` (it's under `backend`, so yes) ? already verified clean.

Let me verify the pre-existing state: was there a `len()` mypy error on these script lines before the PR? Let me check origin/master's version of run_phase3_eval.py around the old line.

43. user (2026-06-18T02:03:32.720Z)

```
seg_cls = segmenter_registry.get(args.segmenter)
if not seg_cls:
    print(f"ERROR: segmenter '{args.segmenter}' not found. "
          f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
    return 2
segmenter = seg_cls(db, cfg)

# Filter entries.
targets = []
for e in entries:
    if args.only and e["video_id"] not in args.only:
=== mypy on master's scripts (segs was Result, len() would also error) ===
checking: does master version also have len-related mypy issues
```

44. assistant (2026-06-18T02:03:45.012Z)

The errors on lines 106 and 150 (`AppConfig` arg-type, `None` union-attr) are clearly pre-existing structural patterns. Line 155's `len(segs)` error stems from `unwrap\_or\_failure`'s return type being `Tuple[Optional[Any], ...]` ? `segs` is `Any | None`, so mypy flags `len(Any | None)`. On master, `segs` was the raw `Result` (`Success | Failure`), and `len(Result)` would also have been a mypy error there. Since `scripts/` isn't in CI's mypy scope either way, this is informational, not a blocker.

I have everything I need. Let me summarize my adversarial findings.

Summary of verification:

- **Bug premise confirmed**: `motion.py` (the default segmenter) returns `Success`/`Failure`; old `len(segs)` / `if not segments` on a raw `Success` crashes with `TypeError`. Fix is real.
- **Helper faithfully mirrors** the inline unwrap at `backend/main.py:201,213`.
- **Edge cases all handled correctly**: `Failure` ? `(None, f)`; `Success(value=[])` ? `([], None)` (flows to empty-guard as a list, verified at runtime); bare list ? `(list, None)`; `Failure` strict-isinstance.
- **Both script call sites fixed**; `tools/generate\_highlights.py` already had its own correct inline handling (4th copy ? DRY nit, out of scope, not buggy).
- **New tests pass (4/4)**; ruff clean; mypy clean on `backend/results.py` (the only changed file in CI's `mypy backend training` scope).
- **The one full-suite test failure** (`test\_fusion\_hybrid::test\_gate\_a\_never\_emptyies\_a\_video`) is a pre-existing Windows test-isolation artifact (passes in isolation; `FileExistsError` from leftover `output/chunks\_gemini\_handoff`), in untouched code ? not introduced by this PR.

- ****Lens probe on OperationalError****: the only ``OperationalError`` handler is ``database.py:40``, a broad ``except: pass`` with no message-text matching, untouched by this PR ? the message-stability concern doesn't apply here.

No must-fix issues found. The PR is safe to merge.

45. user (2026-06-18T02:03:58.292Z)

Structured output provided successfully

46. assistant (2026-06-18T02:04:05.754Z)

Review complete. The PR is a real, correct bug fix with no must-fix issues ? safe to merge.

Key verifications:

- ****Bug premise confirmed****: ``motion.py`` (the default segmenter) returns ``Success`/`Failure``, so the old ``len()`/truthiness` on a raw ``Success`` genuinely crashed with ``TypeError``.
- ****Helper is faithful****: ``unwrap_or_failure`` exactly mirrors the inline `unwrap` at ``backend/main.py:201,213``.
- ****All lens edge cases pass at runtime****: ``Failure ? (None, f)``, empty ``Success(value=[]) ? ([], None)`` (reaches the empty-guard as a list, as intended), bare list passes through.
- ****OperationalError concern doesn't apply****: the only handler (``database.py:40``) is a broad ``except: pass`` with no message-text matching, untouched by this PR.
- ****Tests green**** (4/4 new), ruff clean, mypy clean on the one CI-gated changed file (``backend/results.py``).
- The single full-suite failure is a ****pre-existing Windows test-isolation artifact**** in untouched code, not introduced here.

Two nits only (non-blocking): ``tools/generate_highlights.py:167`` is a fourth inline copy of the `unwrap` that could fold onto the new helper later, and a scripts-only mypy ``len(Any|None)`` note that CI doesn't gate.